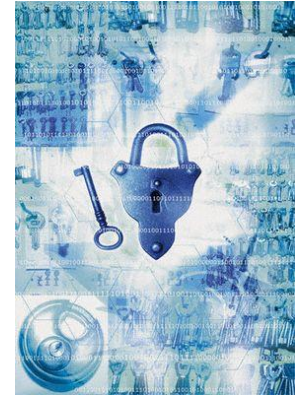


Information Security

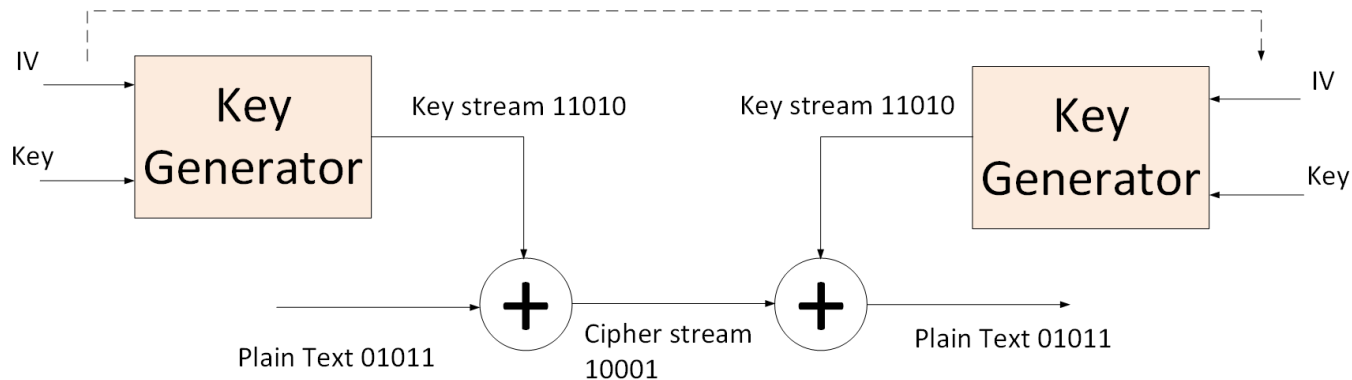


Symmetric Key Crypto

AI컴퓨터공학부
김 희 열

Symmetric Key Crypto

- Stream cipher — like a one-time pad
 - Key is relatively short
 - Key is stretched into a long **key stream**
 - Key stream is then used like a one-time pad



- Block cipher — based on codebook concept
 - Block cipher key determines a codebook
 - Each key yields a different codebook
 - Employ both “confusion” and “diffusion”

Stream Ciphers

- Not as popular today as block ciphers
- We'll discuss two examples
- A5/1
 - Based on shift registers
 - Used in GSM mobile phone system
- RC4
 - Based on a changing lookup table
 - Used many places

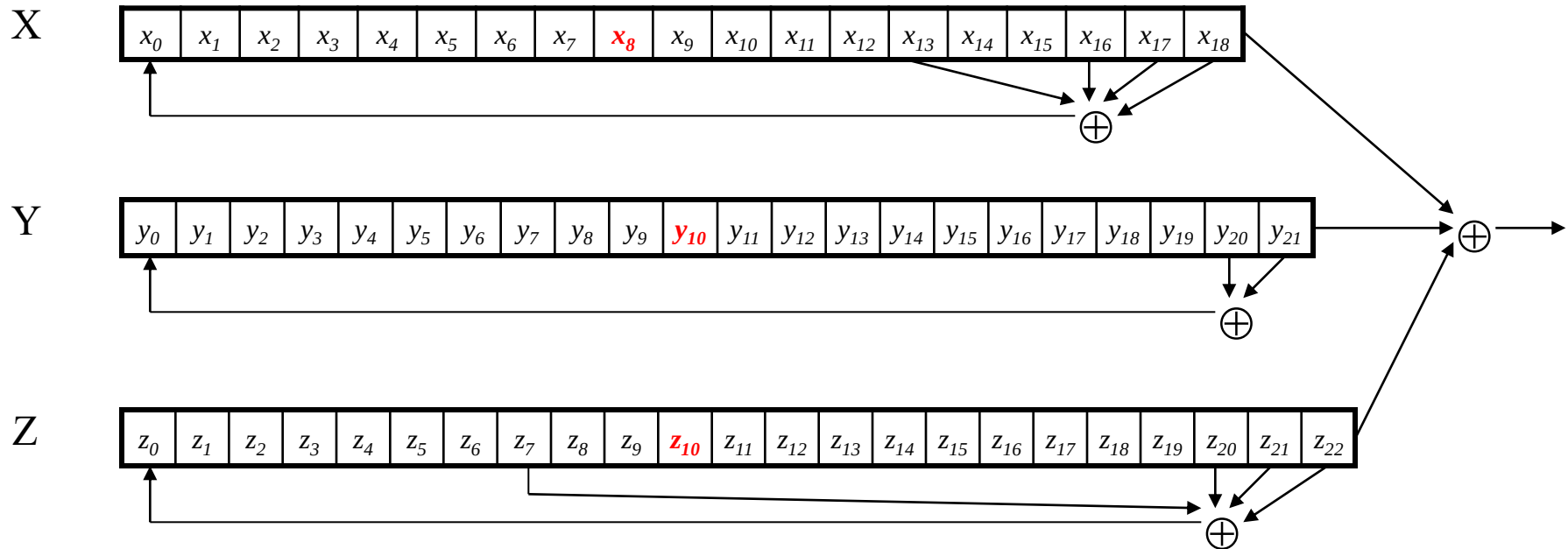
A5/1

- A5/1 consists of 3 *linear feedback shift registers (LFSR)*
 - X: 19 bits ($x_0, x_1, x_2, \dots, x_{18}$)
 - Y: 22 bits ($y_0, y_1, y_2, \dots, y_{21}$)
 - Z: 23 bits ($z_0, z_1, z_2, \dots, z_{22}$)
- Key K
 - 64bits
 - Initially fill the X, Y, Z registers

A5/1

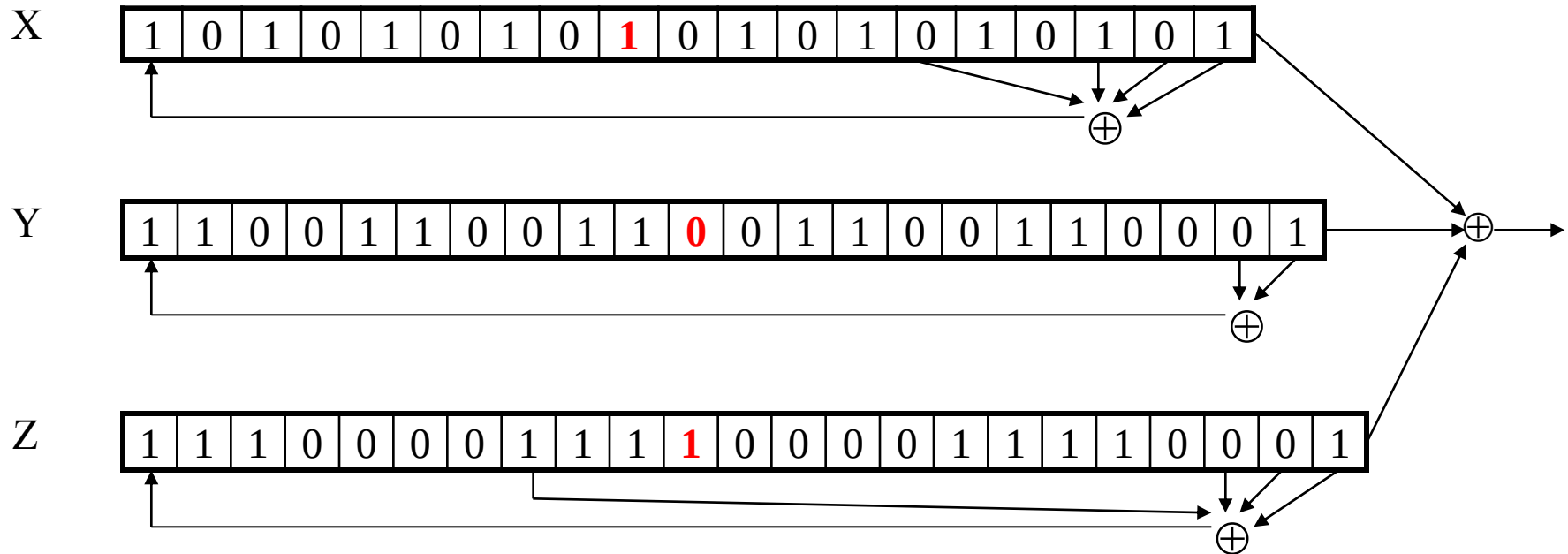
- At each step: $m = \text{maj}(x_8, y_{10}, z_{10})$
 - Choose the majority of x, y, z
 - Examples: $\text{maj}(0,1,0) = 0$ and $\text{maj}(1,1,0) = 1$
- If $x_8 = m$ then X steps
 - $t = x_{13} \oplus x_{16} \oplus x_{17} \oplus x_{18}$
 - $x_i = x_{i-1}$ for $i = 18, 17, \dots, 1$ and $x_0 = t$
- If $y_{10} = m$ then Y steps
 - $t = y_{20} \oplus y_{21}$
 - $y_i = y_{i-1}$ for $i = 21, 20, \dots, 1$ and $y_0 = t$
- If $z_{10} = m$ then Z steps
 - $t = z_7 \oplus z_{20} \oplus z_{21} \oplus z_{22}$
 - $z_i = z_{i-1}$ for $i = 22, 21, \dots, 1$ and $z_0 = t$
- Key stream bit is $x_{18} \oplus y_{21} \oplus z_{22}$

A5/1



- Each value is a single bit
- Key is used as **initial fill** of registers
- Each register steps or not, based on (x_8, y_{10}, z_{10})
- Key stream bit is XOR of right bits of registers
- Then, the key stream bit is XORed with the plaintext bit

A5/1



- In this example, $m = \text{maj}(x_8, y_{10}, z_{10}) = \text{maj}(\mathbf{1}, \mathbf{0}, \mathbf{1}) = \mathbf{1}$
- Register X steps, Y does not step, and Z steps
- Key stream bit is XOR of right bits of registers
- Here, key stream bit will be $0 \oplus 1 \oplus 0 = 1$

Shift Register Crypto

- Shift register-based crypto is efficient in hardware
- Harder to implement in software
- In the past, very popular
- Today, more is done in software due to faster processors
- Shift register crypto still used some

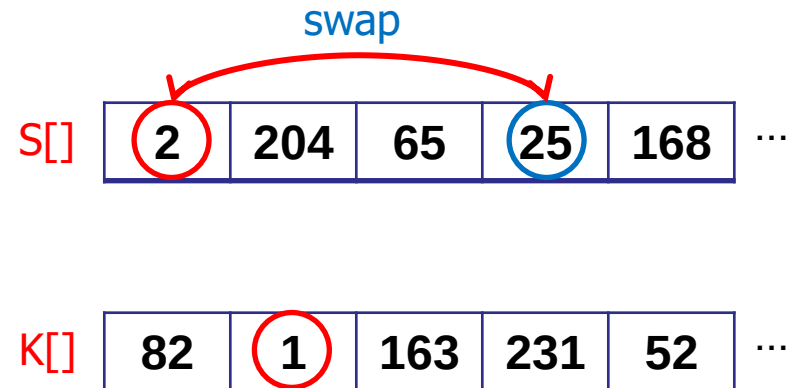
RC4

- A self-modifying lookup table
- Table always contains some permutation of $0, 1, \dots, 255$
- Initialize the permutation using key
 - Key can be of any length from 0 to 256 bytes
- At each step, RC4
 - Swaps elements in current lookup table
 - Selects a key stream **byte** from table
- Each step of RC4 produces a byte
 - Efficient in software
- Each step of A5/1 produces only a bit
 - Efficient in hardware

RC4 Initialization

- $S[]$ is permutation of $0,1,\dots,255$
- $key[]$ contains N bytes of key

```
for i = 0 to 255
    S[i] = i
    K[i] = key[i mod N]
next i
j = 0
for i = 0 to 255
    j = (j + S[i] + K[i]) mod 256
    swap(S[i], S[j])
next j
i = j = 0
```



RC4 Keystream

- For each key stream byte, swap table elements and select byte

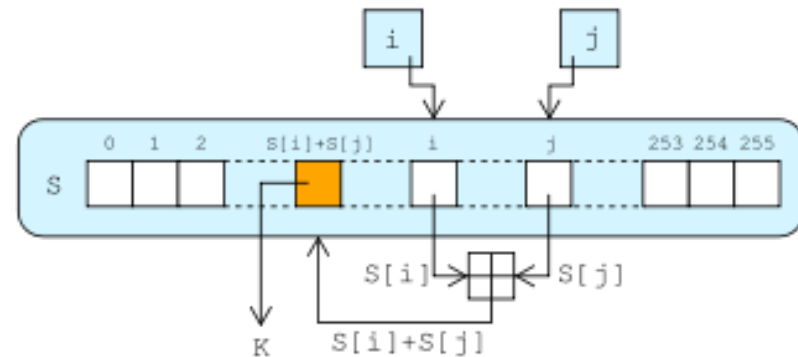
$i = (i + 1) \bmod 256$

$j = (j + S[i]) \bmod 256$

$\text{swap}(S[i], S[j])$ ← Self-modifying

$t = (S[i] + S[j]) \bmod 256$

$\text{keystreamByte} = S[t]$



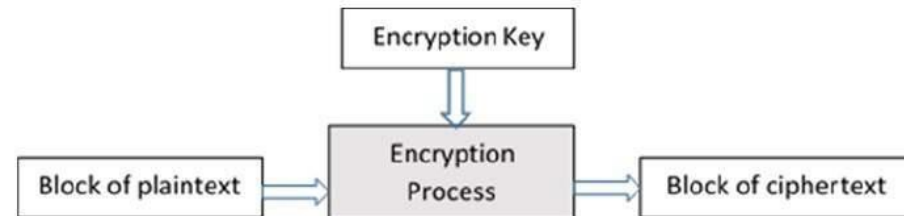
- Use key stream bytes like a one-time pad
- Note:** first 256 bytes must be discarded
 - Otherwise attacker may be able to recover key
- RC4 is used in many apps including WEP, WPA, SSL/TLS
- Optimized for 8-bit processors, not for 32-bit processors
- Prohibited for TLS in 2015 due to the RC4 attack

Stream Ciphers

- Stream ciphers were big in the past
 - Efficient in hardware
 - Speed needed to keep up with voice, etc.
 - Today, processors are fast, so software-based crypto is fast enough
- Future of stream ciphers?
 - Shamir: “the death of stream ciphers”
 - May be exaggerated...

(Iterated) Block Cipher

- Plaintext and ciphertext consists of fixed sized blocks
 - Split plaintext into fixed sized blocks
 - Generate ciphertext blocks from plaintext blocks
- Ciphertext obtained from plaintext by iterating a **round function**
 - This function is applied at each round
- Input to round function consists of
 - Key
 - Output of previous round
- Usually implemented in software



Feistel Cipher

- **Feistel cipher**

- A type of block cipher design
- Not a specific cipher
- Combination of substitution & permutation (transposition)

- Split plaintext block into left and right halves

- Plaintext = (L_0, R_0)

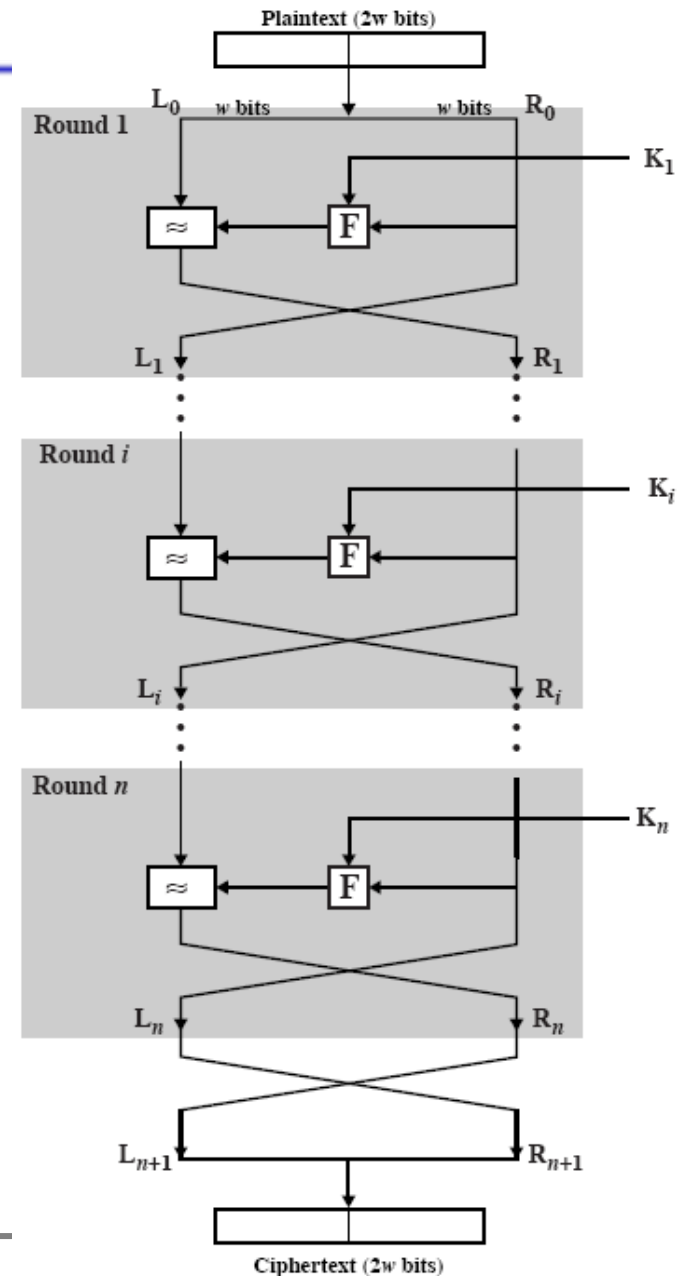
- For each round $i=1,2,\dots,n$, compute

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

where F is **round function** and K_i is **subkey**

- Ciphertext = (L_n, R_n)

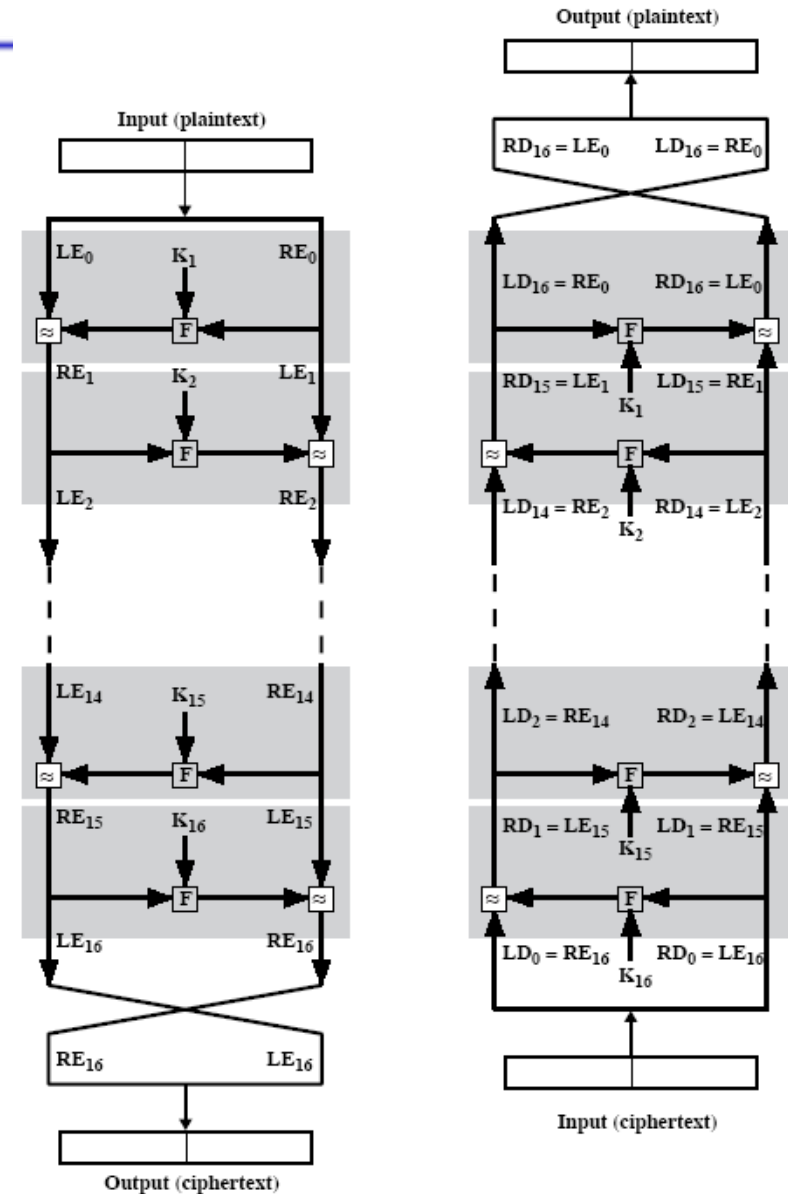


Feistel Cipher

- Decryption: Ciphertext = (L_n, R_n)
- For each round $i=n, n-1, \dots, 1$, compute

$$R_{i-1} = L_i$$

$$L_{i-1} = R_i \oplus F(R_{i-1}, K_i)$$
 where F is round function and K_i is subkey
- Plaintext = (L_0, R_0)
- Formula “works” for any function F
- But only secure for certain functions F
 - If $F(R_{i-1}, K_i) = 0$?



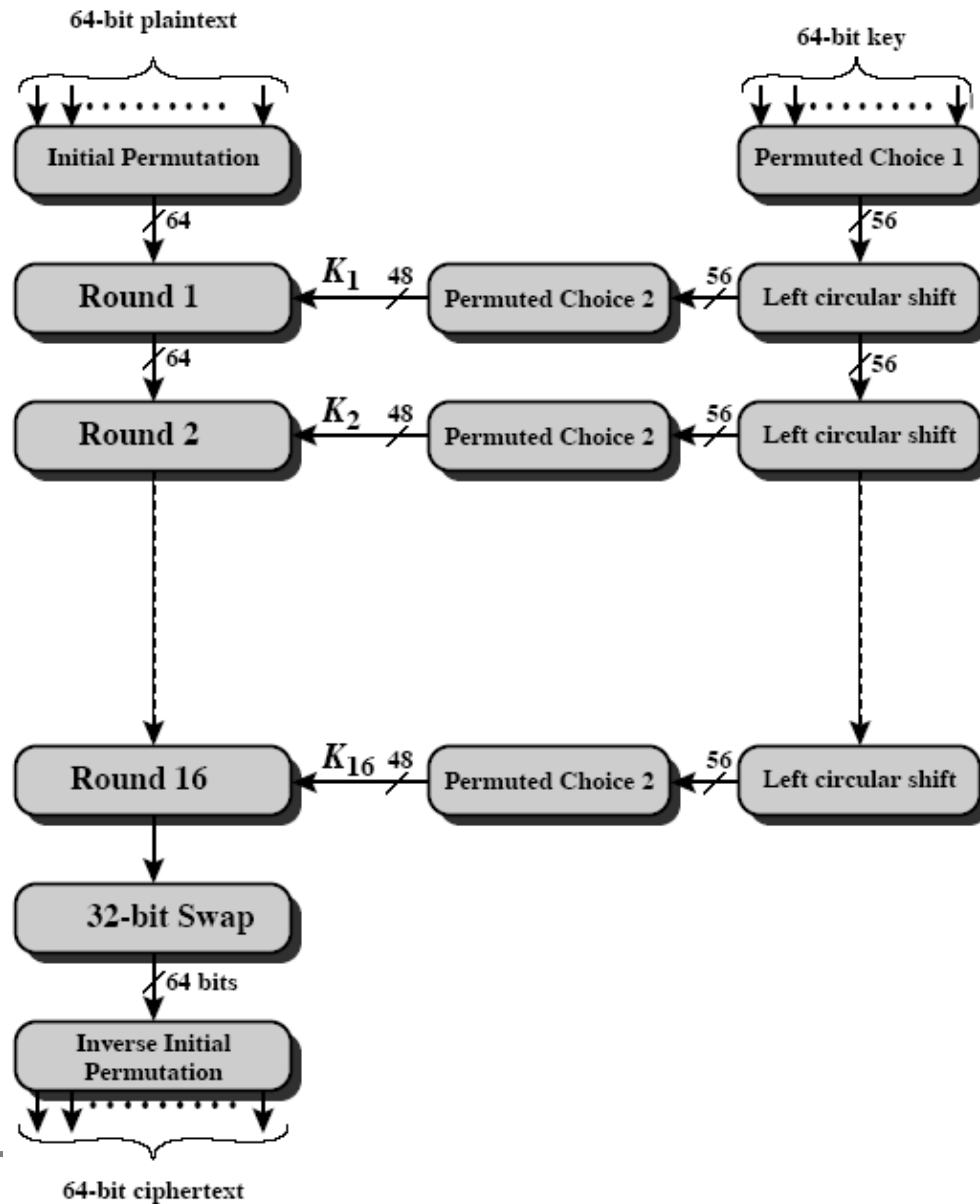
Data Encryption Standard

- DES developed in 1970's
 - Based on IBM Lucifer cipher
- U.S. government standard by NBS (now NIST)
- DES development was controversial
 - NSA (National Security Agency) was secretly involved
 - Design process not open
 - Suspected NSA had placed a [backdoor](#) into DES
 - Key length was reduced
 - From 128 bits to 64bits
 - Subtle changes to Lucifer algorithm

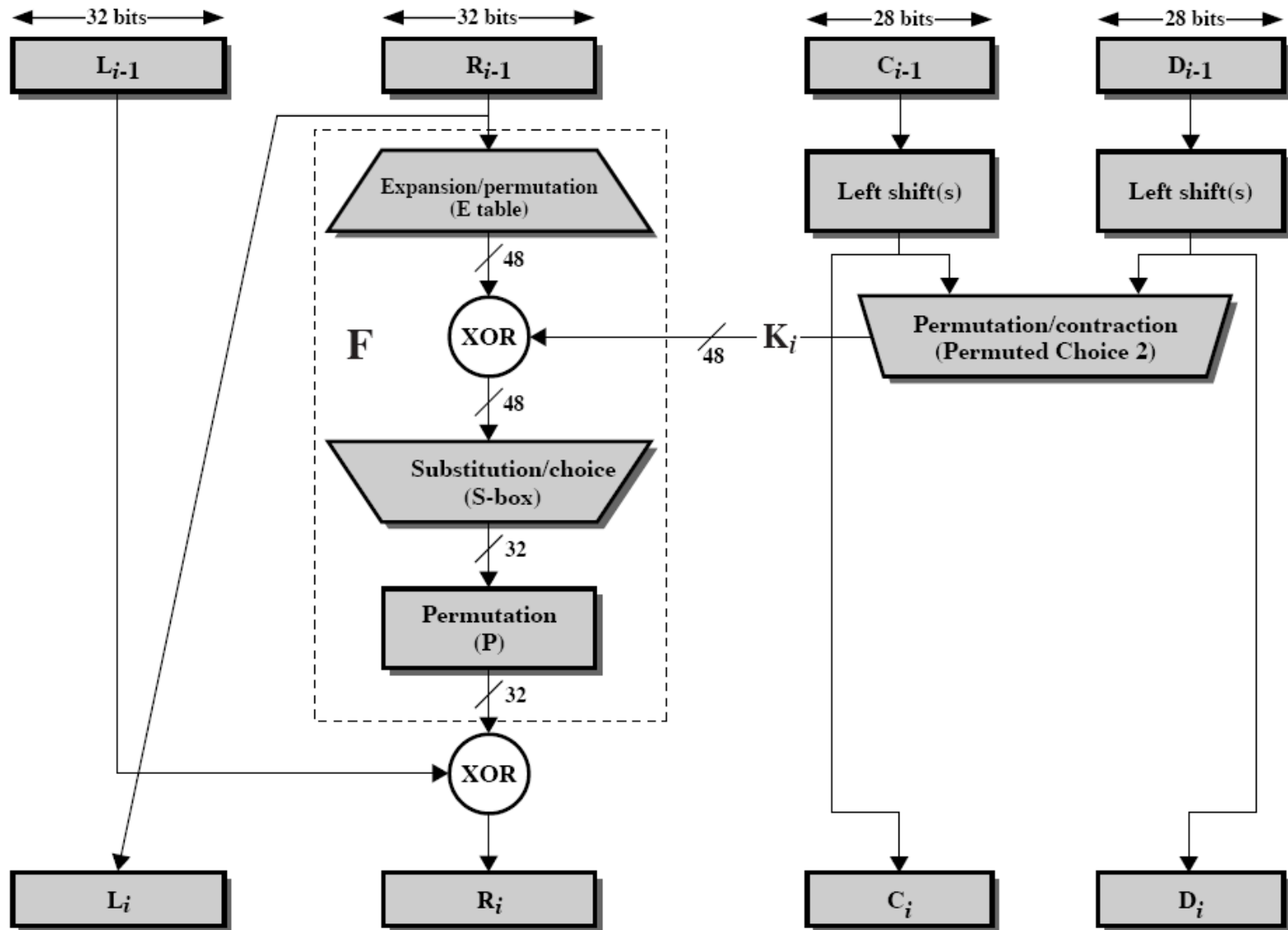
DES Numerology

- DES is a Feistel cipher
 - 64 bit block length
 - 56 bit key length
 - 8 most significant bits from 64 bits are discarded
 - 16 rounds
 - 48 bits of key used each round (subkey)
- Each round is simple (for a block cipher)
- Security depends primarily on “S-boxes”
 - Each S-boxes maps 6 bits to 4 bits

Overall DES Scheme



Single Round of DES

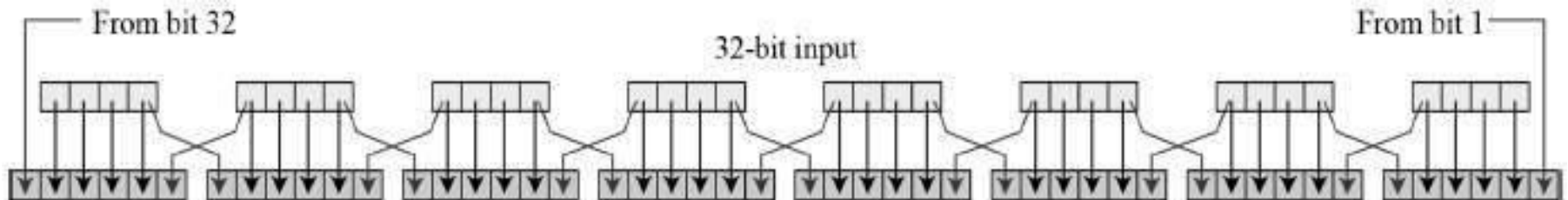


DES Expansion Permutation

- Input 32 bits
- Output 48 bits

(c) Expansion Permutation (E)

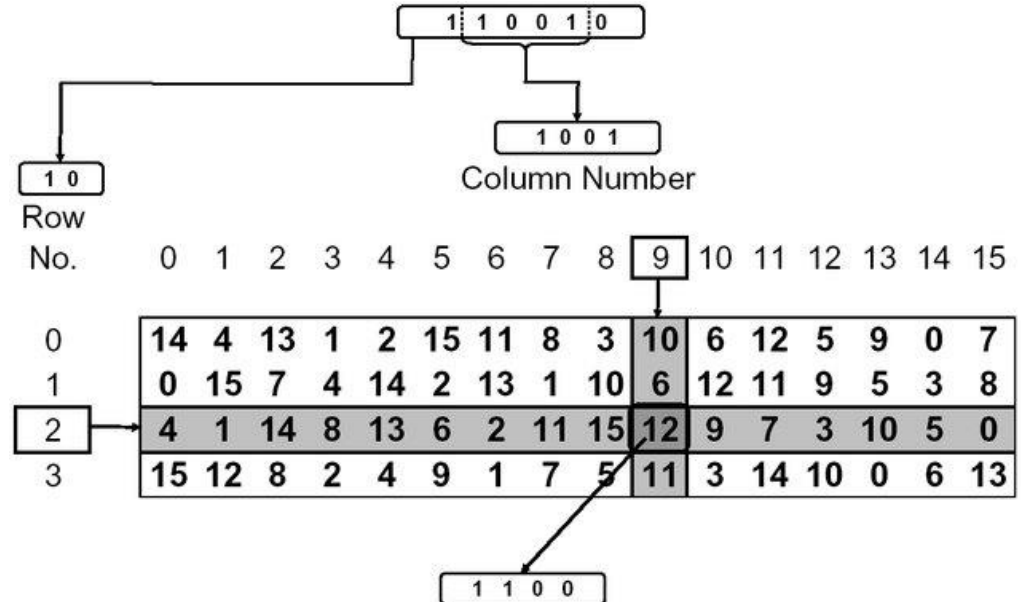
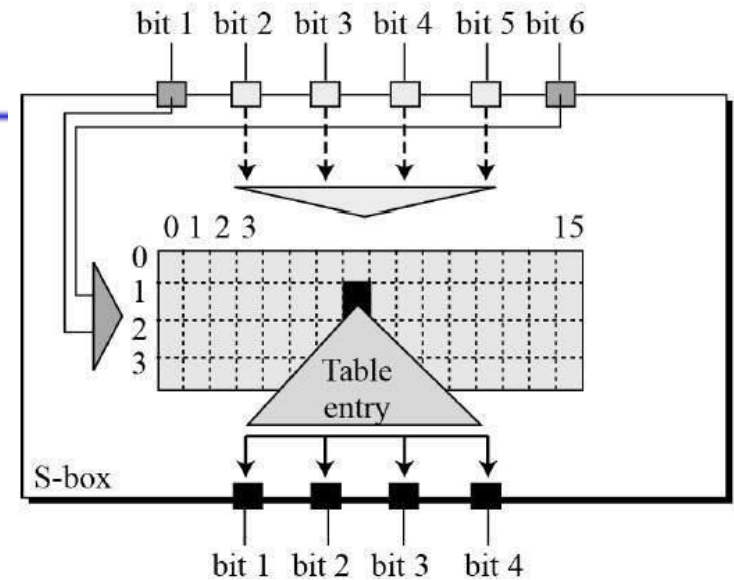
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1



DES S-box

- 8 “substitution boxes” or S-boxes
- Each S-box maps 6 bits to 4 bits

סורה	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S₁																
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	3	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	13	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S₂																
0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S₃																
0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S₄																
0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S₅																
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S₆																
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S₇																
0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S₈																
0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

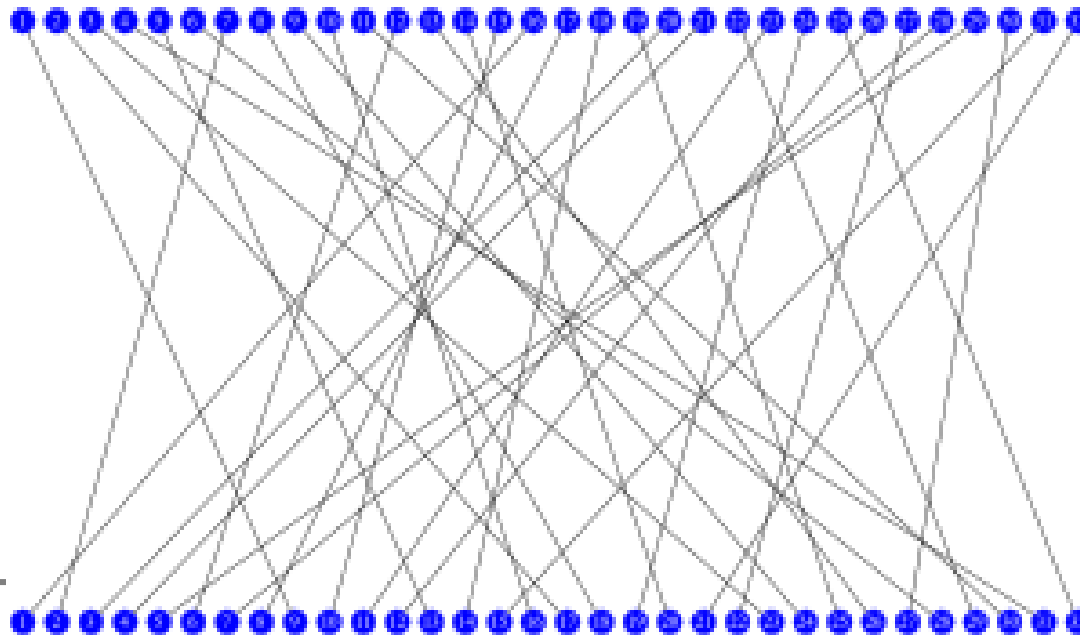


DES P-box

- Input 32 bits
- Output 32 bits

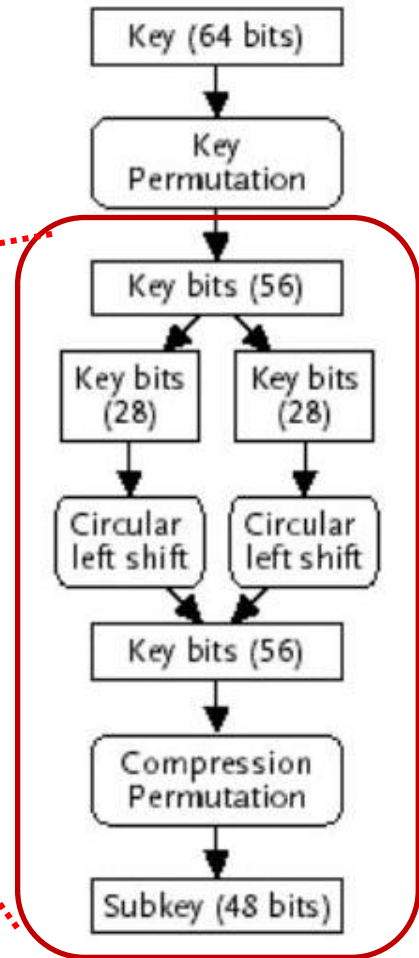
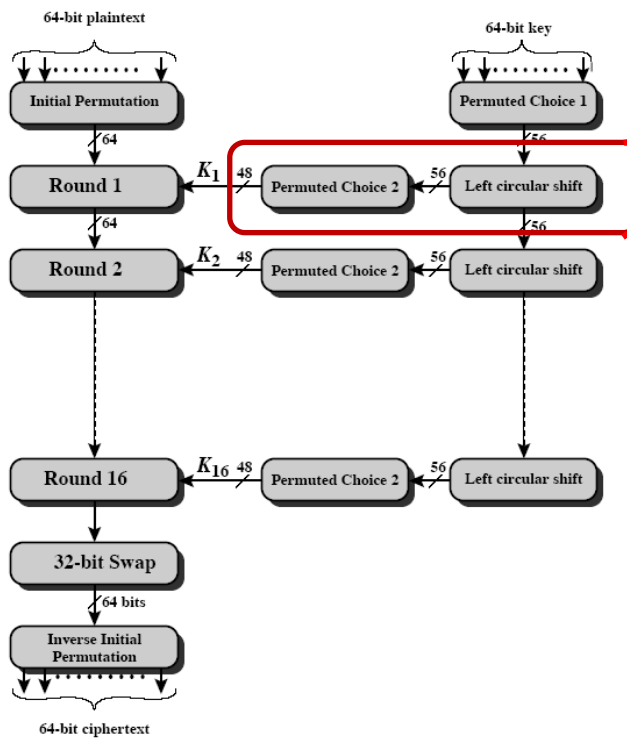
(d) **Permutation Function (P)**

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25



DES Subkey

- 56 bit DES key
- Key schedule
 - 48 bits subkey is generated at each round



Security of DES

- Security of DES depends a lot on S-boxes
 - Everything else in DES is linear
- Thirty years of intense analysis has revealed no “back door”
- Attacks today use exhaustive key search

DES Cracker "Deep Crack"
custom microchip



DES Cracker circuit board
fitted with Deep Crack chips



Block Cipher Notation

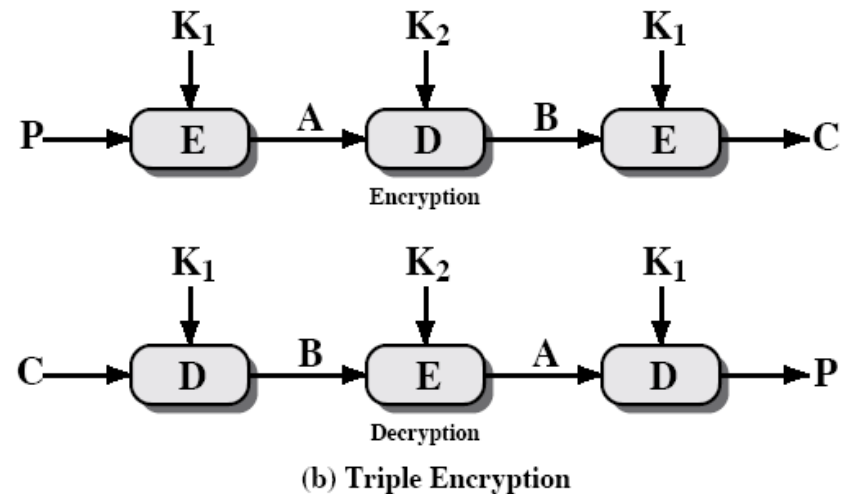
- P = plaintext block
- C = ciphertext block
- Encrypt P with key K to get ciphertext C
 - $C = E(P, K)$
- Decrypt C with key K to get plaintext P
 - $P = D(C, K)$
- Note that
 - $P = D(E(P, K), K)$ and $C = E(D(C, K), K)$

Triple DES

- Today, 56 bit DES key is too small
- But DES is everywhere: What to do?

- **Triple DES** or **3DES** (112 bit key)

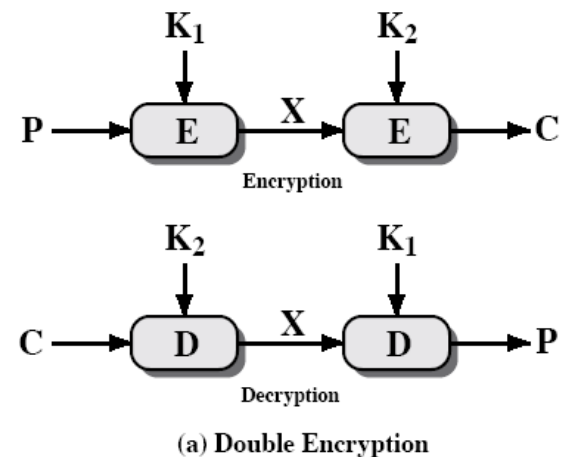
- $C = E(D(E(P, K_1), K_2), K_1)$
- $P = D(E(D(C, K_1), K_2), K_1)$



- Why use Encrypt-Decrypt-Encrypt (EDE) with 2 keys?
 - Backward compatible: $E(D(E(P, K), K), K) = E(P, K)$
 - And 112 bits is enough

3DES

- Why not $C = E(E(P, K), K)$?
 - Still just 56 bit key
- Why not $C = E(E(P, K_1), K_2)$?



- A (semi-practical) known plaintext attack ([Meet-in-the-Middle attack](#))
 - Precompute table of $E(P, K_1)$ for every possible key K_1 (resulting table has 2^{56} entries) & sort them
 - Then for each possible K_2 compute $D(C, K_2)$ until a match in table is found
 - When match is found, have $E(P, K_1) = D(C, K_2)$
 - Result is keys: $C = E(E(P, K_1), K_2)$

DES History

- Chronology http://en.wikipedia.org/wiki/Data_Encryption_Standard
 - 1973: NBS publishes a first request for a standard encryption algorithm
 - 1974: NBS publishes a second request for encryption algorithms
 - 1975: DES is published in the Federal Register for comment
 - 1976: First and second workshop on DES
 - 1976: DES is approved as a standard
 - 1977: DES is published as a FIPS standard FIPS PUB 46
 - 1983: DES reaffirmed for the first time
 - 1986: Videocipher II, a TV satellite scrambling system based upon DES begins use by HBO
 - 1988: DES is reaffirmed for the second time as FIPS 46-1, superseding FIPS PUB 46
 - 1992: Biham and Shamir publish the first theoretical attack with less complexity than brute force: differential cryptanalysis. However, it requires an unrealistic 2^{47} chosen plaintexts
 - 1993: DES is reaffirmed for the third time as FIPS 46-2

DES History

- 1994: The first experimental cryptanalysis of DES is performed using linear cryptanalysis (Matsui, 1994)
- 1997: The DESCHALL Project breaks a message encrypted with DES for the first time in public
- 1998: The Electronic Frontier Foundation (EFF)'s DES cracker (Deep Crack) breaks a DES key in 56 hours
- 1999: Together, Deep Crack and distributed.net break a DES key in 22 hours and 15 minutes
- 1999: DES is reaffirmed for the fourth time as FIPS 46-3, which specifies the preferred use of Triple DES, with single DES permitted only in legacy systems
- 2001: The Advanced Encryption Standard is published in FIPS 197
- 2002: The AES standard becomes effective
- 2004: The withdrawal of FIPS 46-3 (and a couple of related standards) is proposed in the Federal Register
- 2005: NIST withdraws FIPS 46-3
- 2016: GeForce GTX 1080 Ti recovers DES key in 15 days

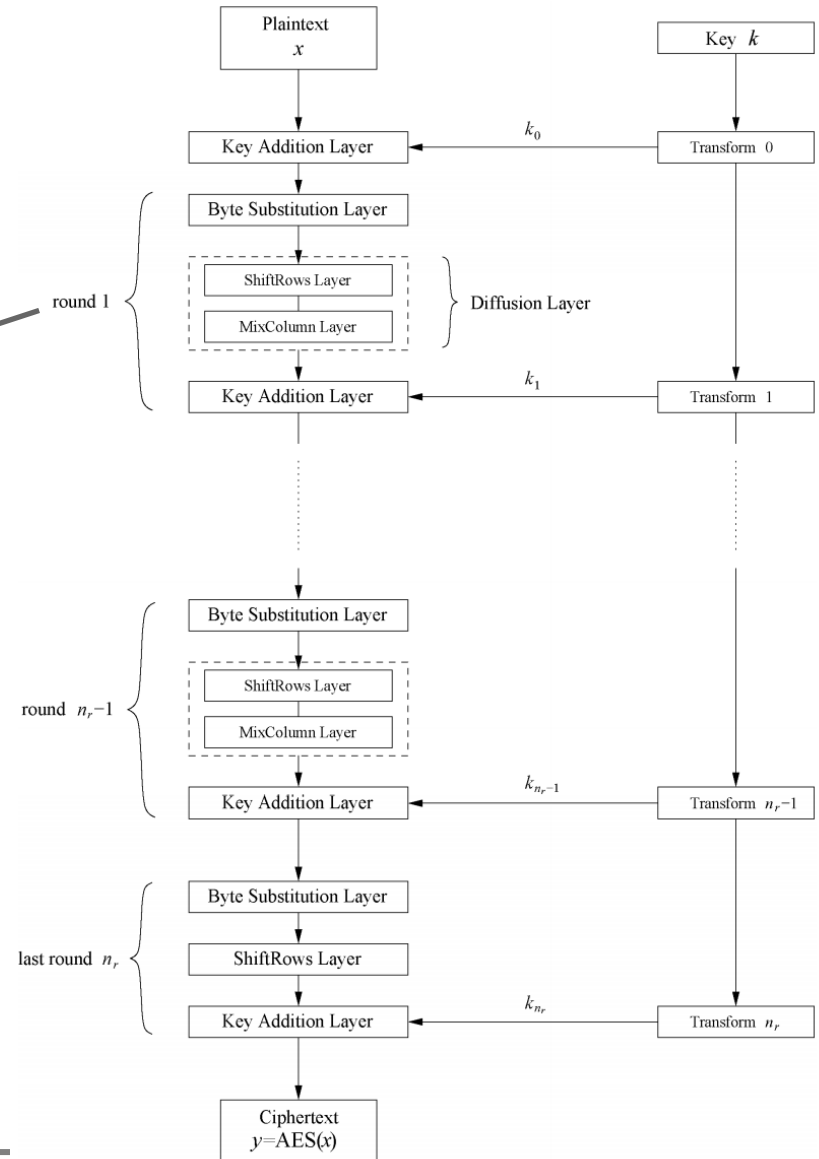
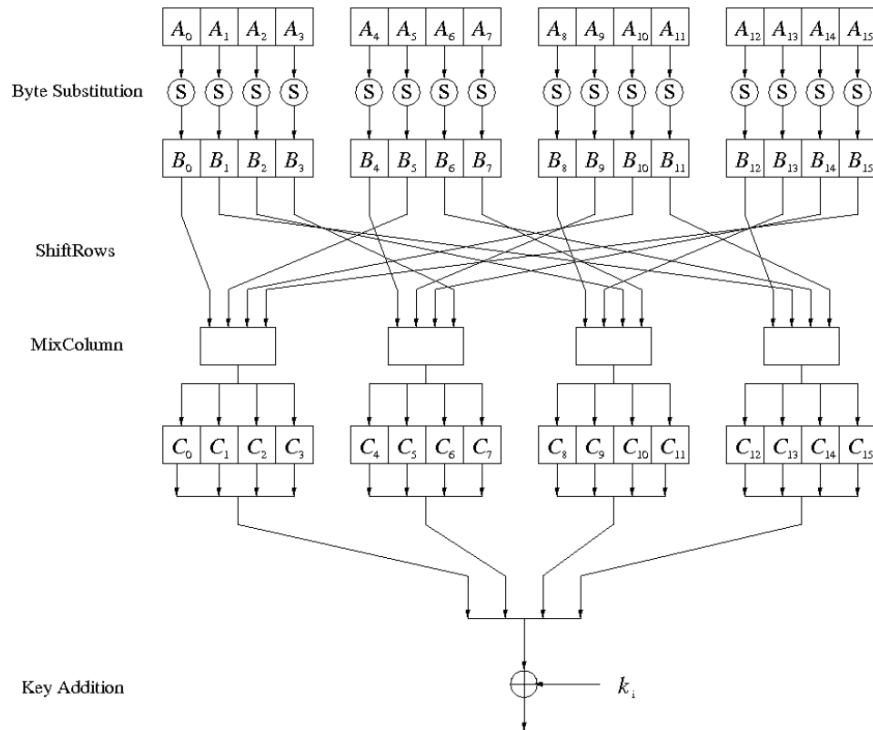
Advanced Encryption Standard

- Replacement for DES
- AES competition (late 90's)
 - NSA openly involved
 - Transparent process
 - Many strong algorithms proposed
 - Rijndael Algorithm ultimately selected
 - Pronounced like “Rain Doll” or “Rhine Doll”
- Iterated block cipher (like DES)
- Not a Feistel cipher (unlike DES)

Advanced Encryption Standard (AES)

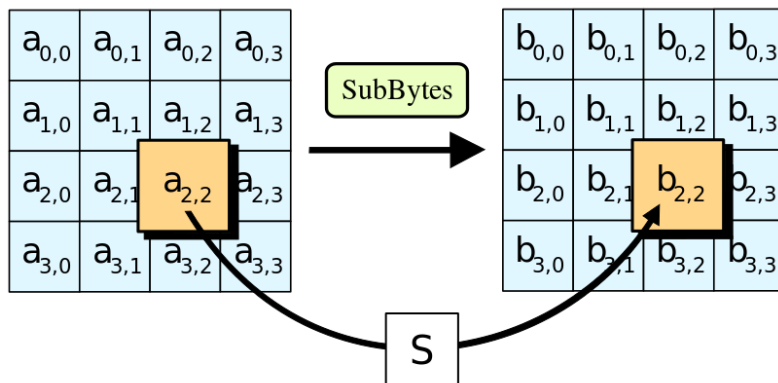
- **Block size:** 128 bits
- **Key length:** 128, 192 or 256 bits
- 10 to 14 rounds (depends on key length)
- Based on Substitution-Permutation Network

AES overview



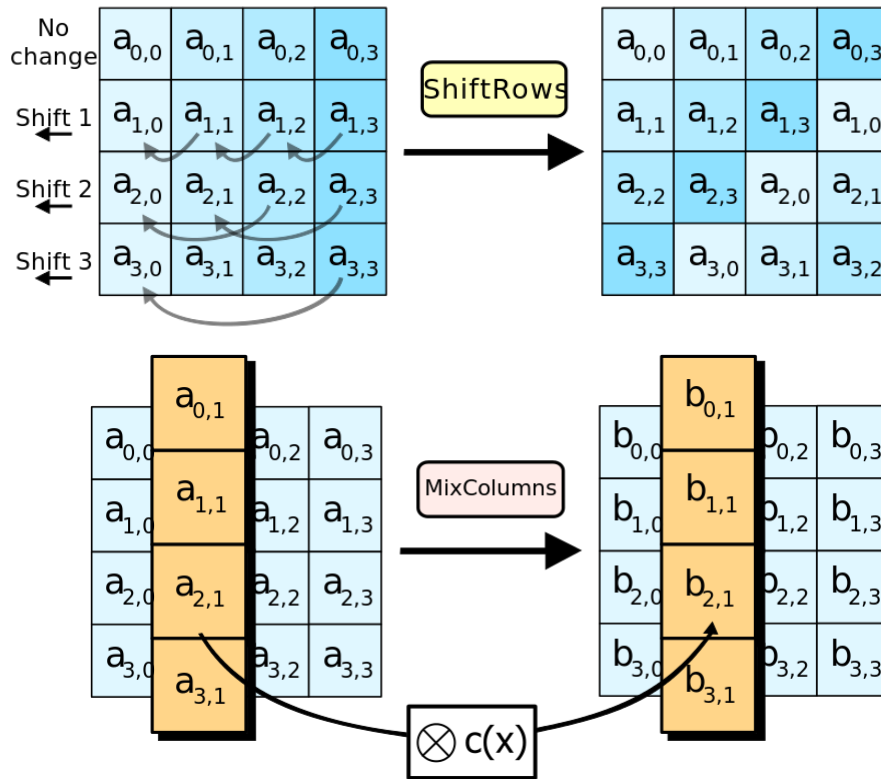
AES SubBytes

- Byte substitution is AES's "S-box"
- Improve confusion
 - Obscure the relationship between plaintext and ciphertext
- The only nonlinear element
 - Computes multiplicative inverse in $GF(2^8)$ on irreducible poly.
 $x^8+x^4+x^3+x+1$
 - Then, bit scrambling $b'_i = b_i \otimes b_{(i+4) \bmod 8} \otimes b_{(i+5) \bmod 8} \otimes b_{(i+6) \bmod 8} \otimes b_{(i+7) \bmod 8} \otimes c_i$
- In software implementations, usually use a 16*16 lookup table

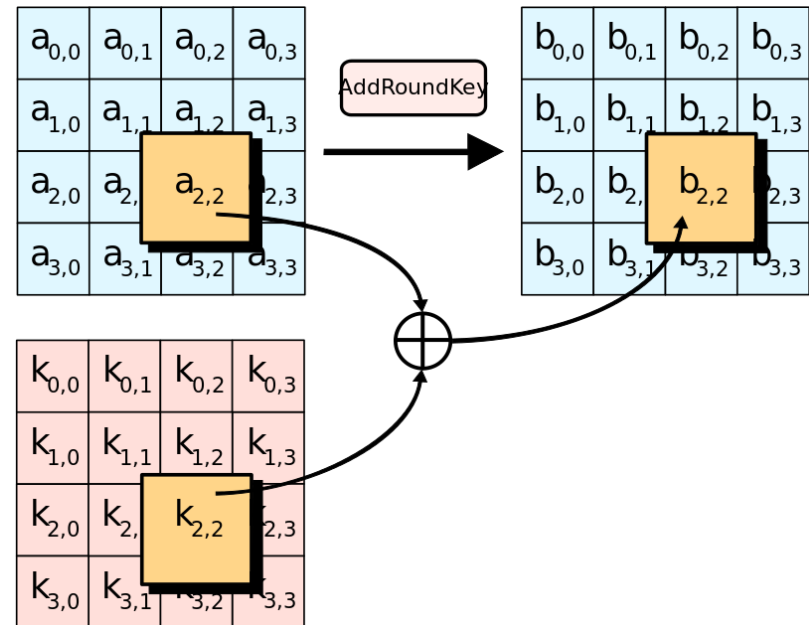


	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

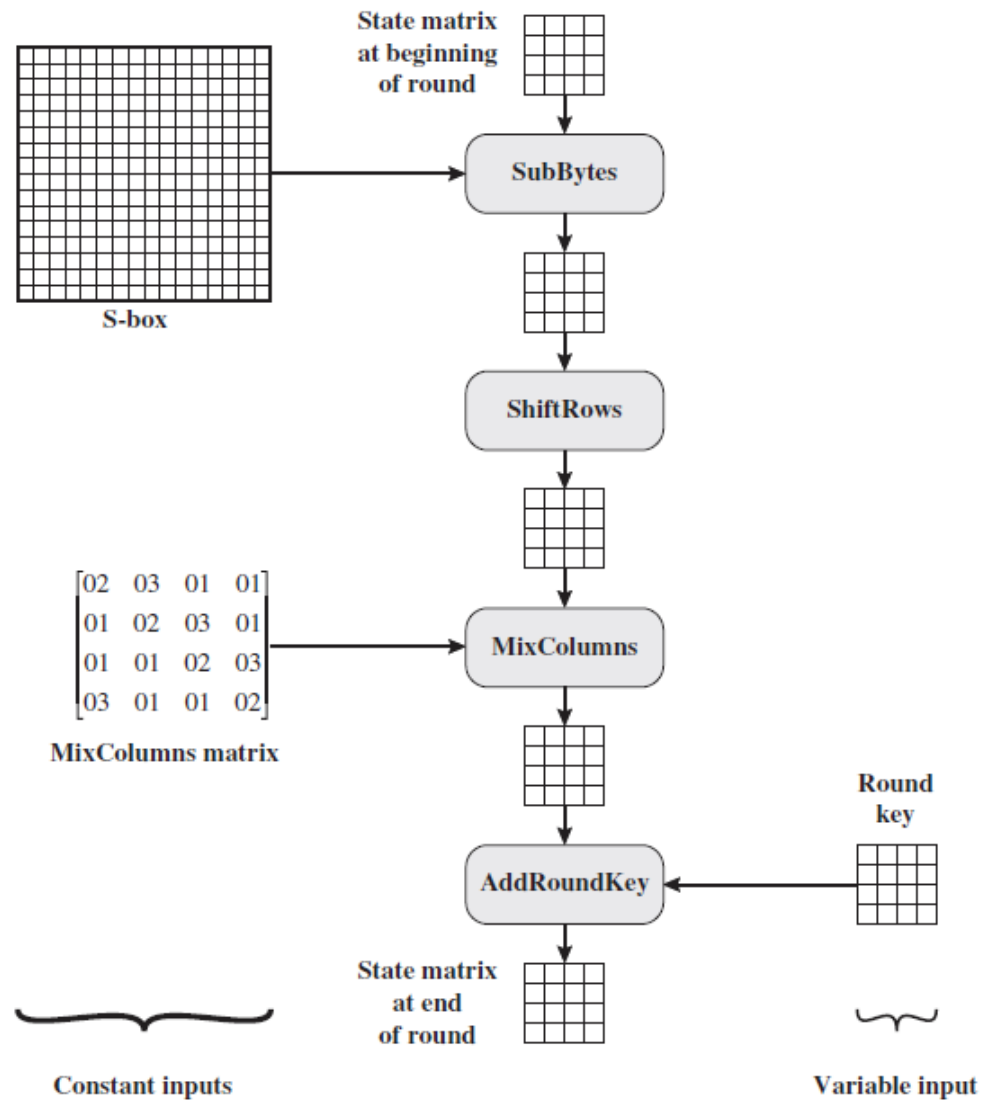
AES other operations



Improve diffusion



AES round



AES process

How to navigate through the animation:

- > press **Control + F** to get into full screen mode
- > use **Enter** key to advance
- > use **Slide controller** on bottom to navigate
- > press **c** to show/hide the slide controller

01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20

From <https://www.youtube.com/watch?v=gP4PgVGudtg>

AES Implementation

- Efficient software implementation is required
- Straightforward impl is well suited for 8-bit processors (e.g., smart cards), but inefficient on 32-bit or 64-bit processors
- A more sophisticated approach
 - Merge all round functions into one lookup table
- Typical SW speeds are more than 1.6Gbit/s on modern 64-bit processors

AES summary

- AES is a modern block cipher supporting 3 key lengths: 128, 192, 256bit
- No attacks have been found better than brute-force
- AES is not based on Feistel cipher
- AES is part of numerous open standards such as IPsec or TLS
- AES is efficient in software and hardware

A Few Other Block Ciphers

- IDEA
- Blowfish
- RC6
- TEA
- Etc...

IDEA

- Invented by James Massey
 - One of the giants of modern crypto
- IDEA has 64-bit block, 128-bit key
- IDEA uses **mixed-mode arithmetic**
- Combine different math operations
 - IDEA the first to use this approach
 - Frequently used today

Blowfish

- Blowfish encrypts 64-bit blocks
- Key is variable length, up to 448 bits
- Invented by Bruce Schneier
- Almost a Feistel cipher
$$R_i = L_{i-1} \oplus K_i$$
$$L_i = R_{i-1} \oplus F(L_{i-1} \oplus K_i)$$
- The round function F uses 4 S-boxes
 - Each S-box maps 8 bits to 32 bits
- **Key-dependent S-boxes**
 - S-boxes determined by the key

RC6

- Invented by Ron Rivest
 - Also invented RSA, RC4, MD5
- Variables
 - Block size
 - Key size
 - Number of rounds
- One of AES finalists
- Uses **data dependent rotations**
 - Unusual to rely on data as part of algorithm

TEA

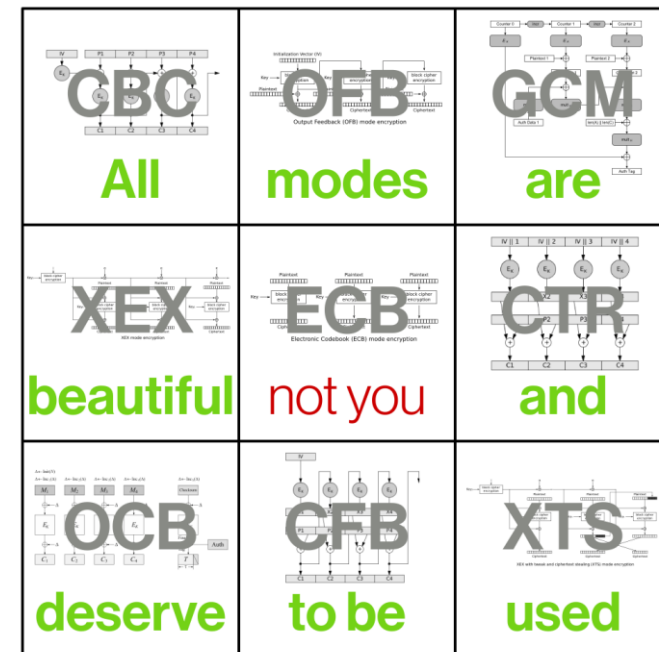
- **Tiny Encryption Algorithm**
- **Almost** a Feistel cipher
 - Uses + and - instead of $\oplus$ (XOR)
- Simple, easy to implement, fast, low memory requirement, etc.
- Possibly a related key attack
- eXtended TEA (XTEA)
 - eliminates related key attack (slightly more complex)
- Simplified TEA (STEAL)
 - insecure version used as an example for cryptanalysis

Block Cipher Modes

- How to encrypt multiple blocks?
- A new key for each block?
 - As bad as (or worse than) a one-time pad!
- Encrypt each block independently?
- Make encryption depend on previous block(s), i.e., “chain” the blocks together?
- How to handle partial blocks?

Modes of Operation

- Many modes of operation — we discuss three
- Electronic Codebook (**ECB**) mode
 - Obvious thing to do
 - Encrypt each block independently
 - There is a serious weakness
- Cipher Block Chaining (**CBC**) mode
 - Chain the blocks together
 - More secure than ECB, virtually no extra work
- Counter Mode (**CTR**) mode
 - Acts like a stream cipher
 - Popular for random access



ECB Mode

- Notation: $C=E(P,K)$
- Given plaintext $P_1, P_2, \dots, P_n$

- Obvious way to use a block cipher is

Encrypt

$$C_1 = E(P_1, K),$$

$$C_2 = E(P_2, K),$$

$$C_3 = E(P_3, K), \dots$$

Decrypt

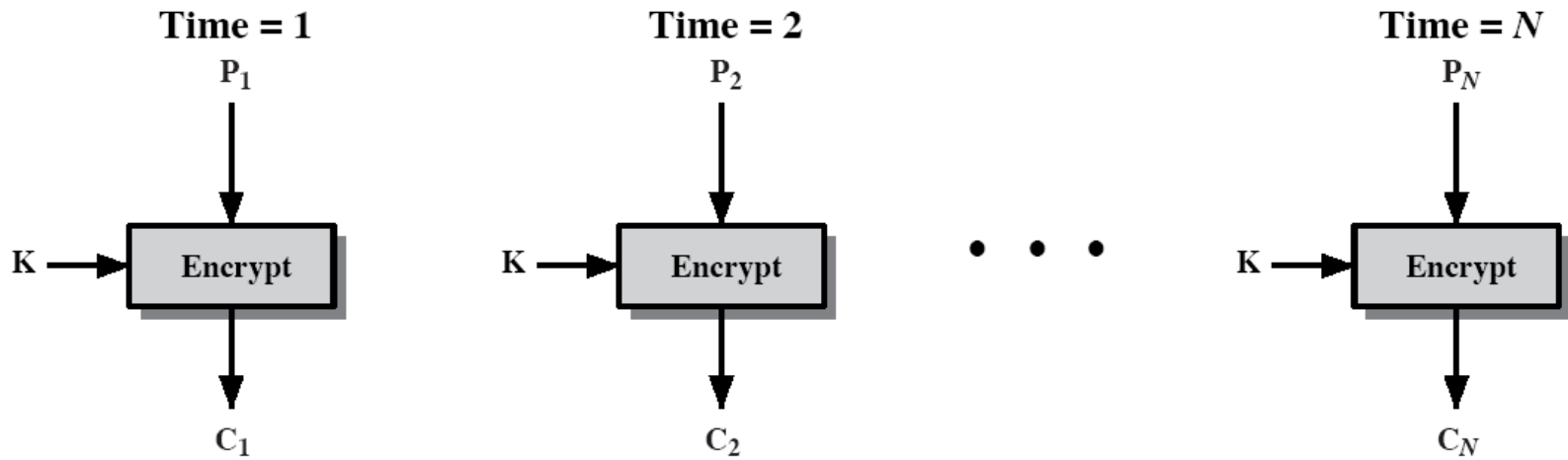
$$P_1 = D(C_1, K),$$

$$P_2 = D(C_2, K),$$

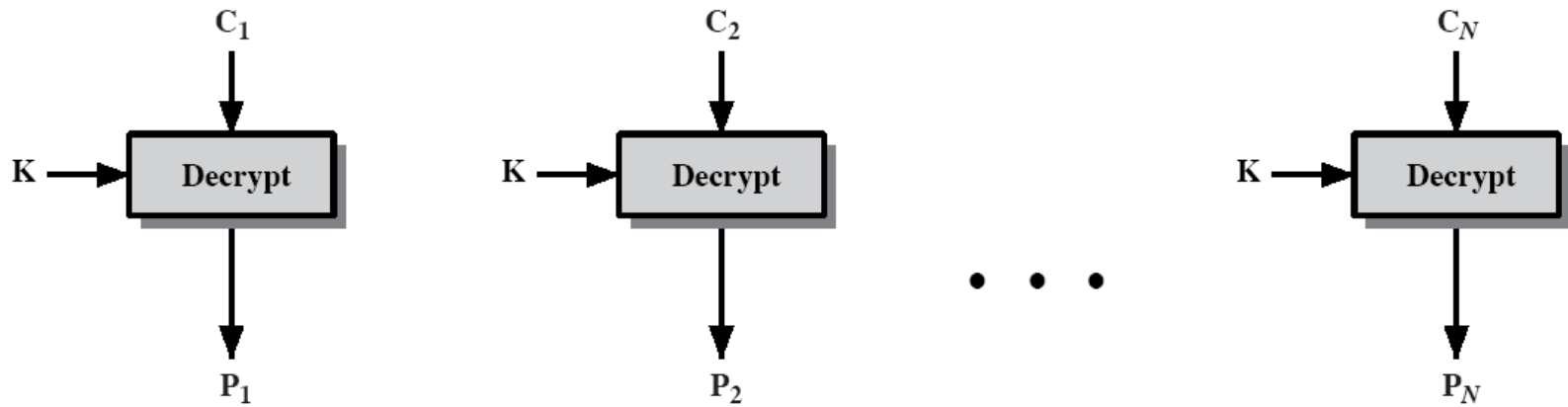
$$P_3 = D(C_3, K), \dots$$

- For a fixed key K , this is an electronic version of a codebook cipher
- A new codebook for each key

ECB Mode



(a) Encryption



(b) Decryption

ECB Cut and Paste Attack

- Suppose plaintext is

Alice digs Bob. Trudy digs Tom.

- Assuming 64-bit blocks and 8-bit ASCII:

$P_1 = \text{"Alice di"},$ $P_2 = \text{"gs Bob. "},$
 $P_3 = \text{"Trudy di"},$ $P_4 = \text{"gs Tom. "}$

- Ciphertext:

C_1, C_2, C_3, C_4

- Trudy cuts and pastes:

C_1, C_4, C_3, C_2

- Decrypts as

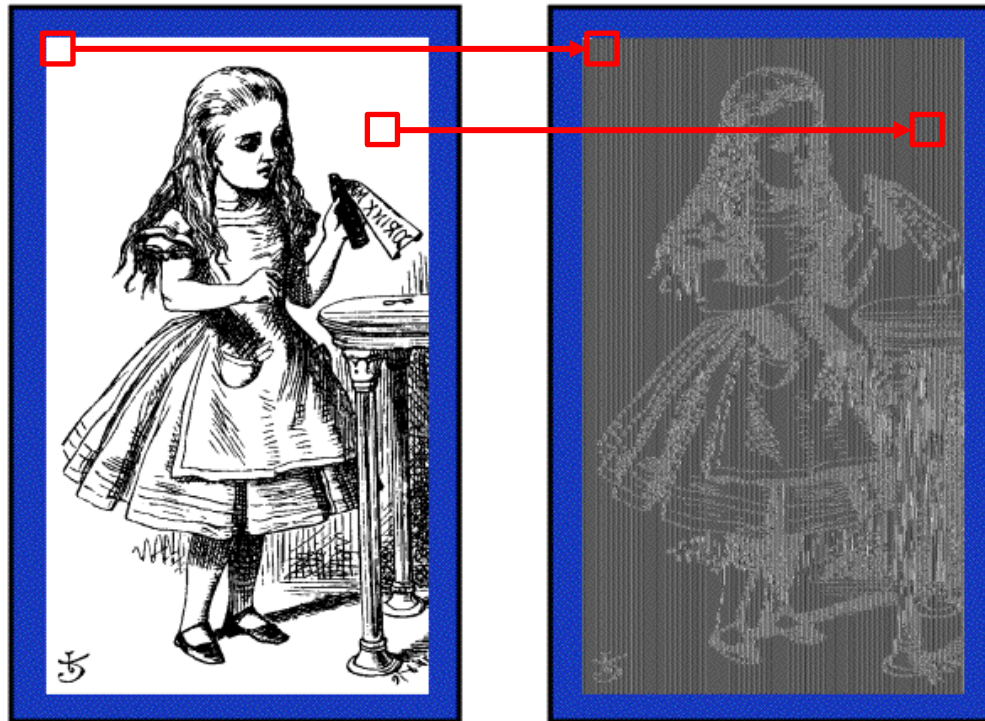
Alice digs Tom. Trudy digs Bob.

ECB Weakness

- Suppose $P_i = P_j$
- Then $C_i = C_j$ and Trudy knows $P_i = P_j$
- This gives Trudy some information, even if she does not know P_i or P_j
- Trudy might know P_i
- Is this a serious issue?

Alice Hates ECB Mode

- Alice's uncompressed image, Alice ECB encrypted



- Why does this happen?
 - Same plaintext block $\rightarrow$ same ciphertext!

CBC Mode

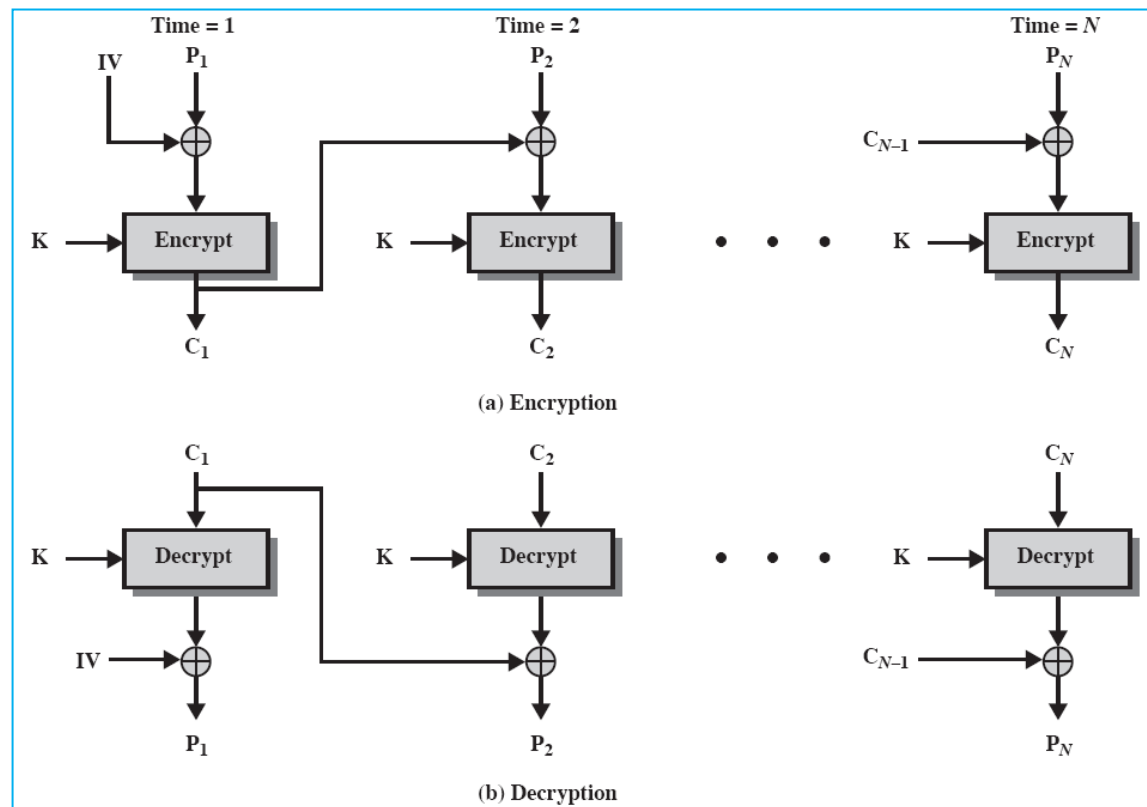
- Blocks are “chained” together
- A random initialization vector(IV) is required to initialize CBC mode
- IV is random, but need not be secret

Encryption

$$\begin{aligned}C_1 &= E(IV \oplus P_1, K), \\C_2 &= E(C_1 \oplus P_2, K), \\C_3 &= E(C_2 \oplus P_3, K), \dots\end{aligned}$$

Decryption

$$\begin{aligned}P_1 &= IV \oplus D(C_1, K), \\P_2 &= C_1 \oplus D(C_2, K), \\P_3 &= C_2 \oplus D(C_3, K), \dots\end{aligned}$$

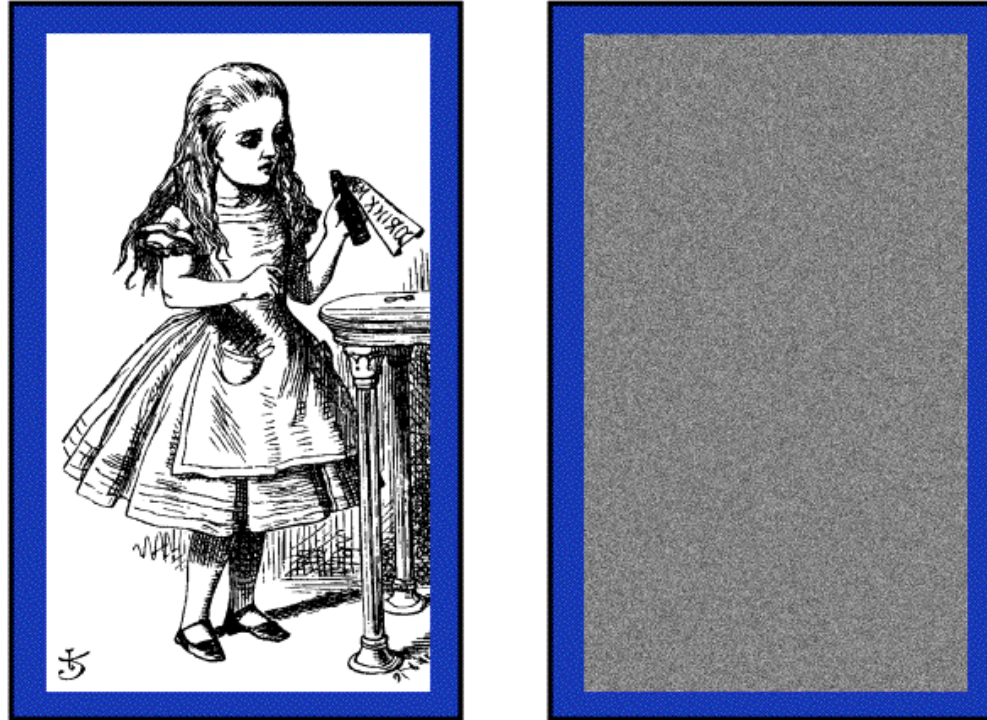


CBC Mode

- Identical plaintext blocks yield different ciphertext blocks
- Cut and paste is still possible, but more complex
 - will cause garbles
- If C_2 is garbled to, say, G then
$$P_2 \neq C_1 \oplus D(G, K), P_3 \neq G \oplus D(C_3, K)$$
- But $P_4 = C_3 \oplus D(C_4, K), \dots$
- Automatically recovers from errors!

Alice Likes CBC Mode

- Alice's uncompressed image, Alice CBC encrypted



- Why does this happen?
 - Same plaintext yields different ciphertext!

Counter Mode (CTR)

- CTR is popular for random access
- Can use block cipher like stream cipher

Encryption

$$C_1 = P_1 \oplus E(\text{Counter}, K),$$

$$C_2 = P_2 \oplus E(\text{Counter} + 1, K),$$

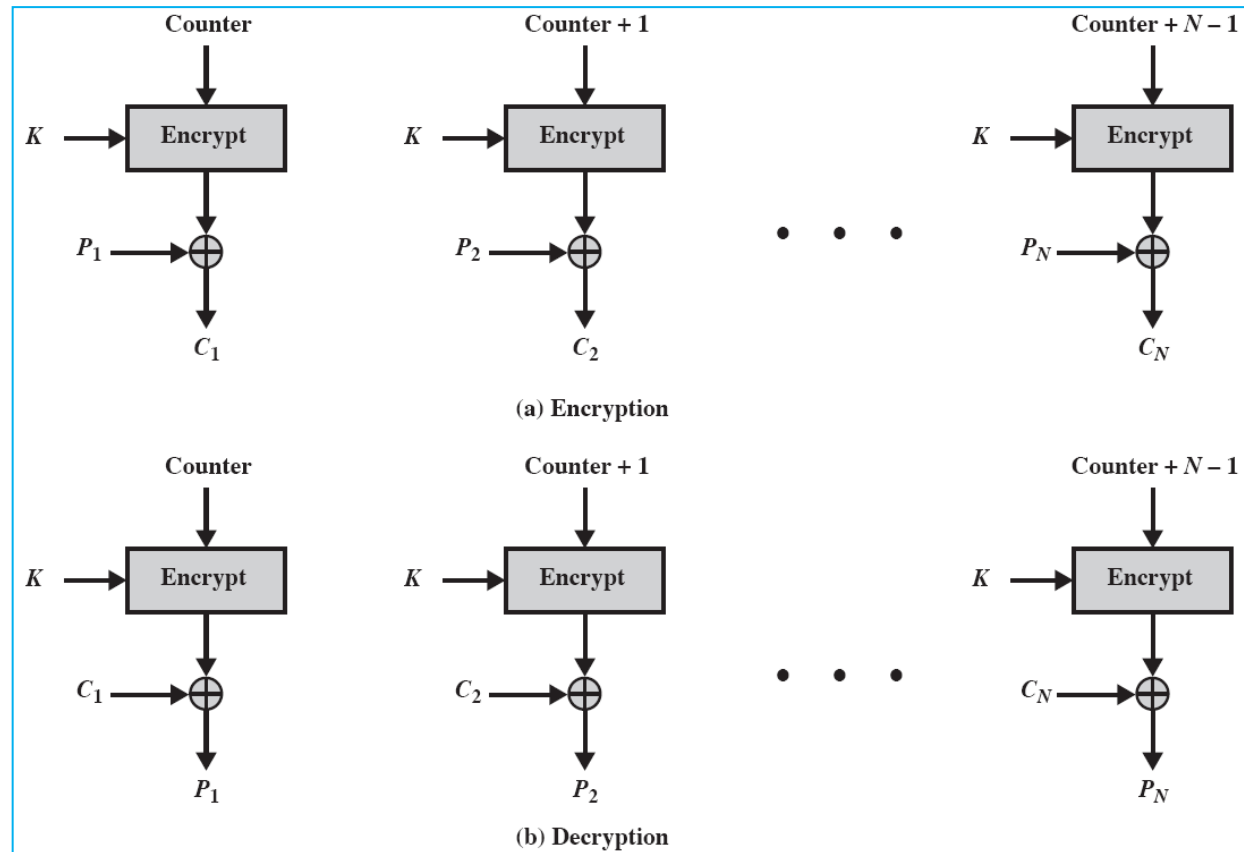
$$C_3 = P_3 \oplus E(\text{Counter} + 2, K),$$

Decryption

$$P_1 = C_1 \oplus E(\text{Counter}, K),$$

$$P_2 = C_2 \oplus E(\text{Counter} + 1, K),$$

$$P_3 = C_3 \oplus E(\text{Counter} + 2, K),$$

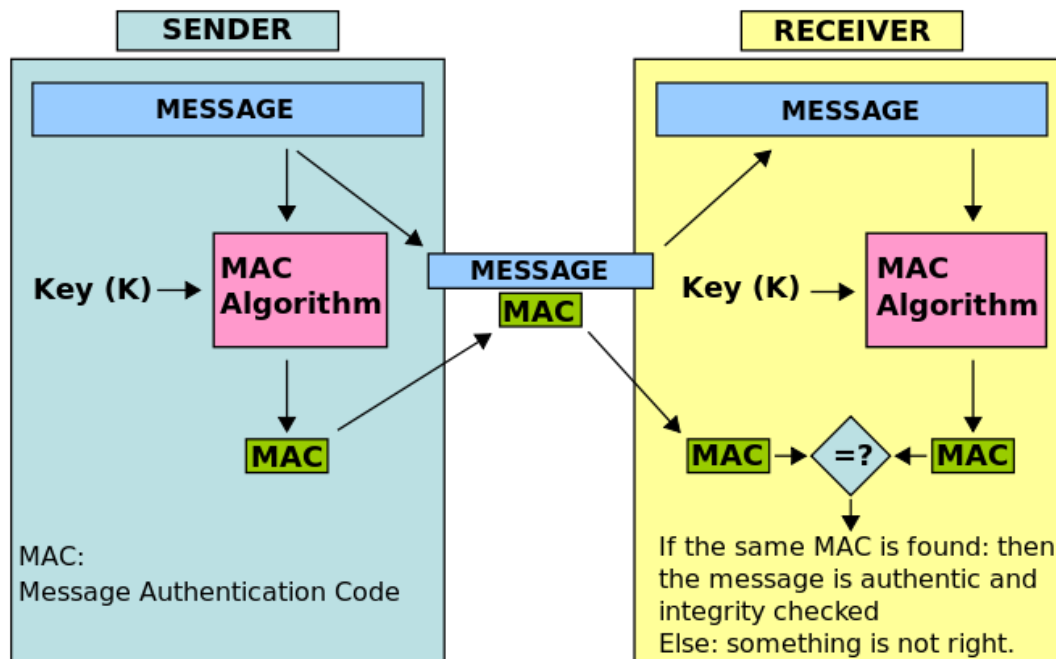


Data Integrity

- **Integrity**
 - prevent (or at least detect) unauthorized modification of data
- Example: Inter-bank fund transfers
 - Confidentiality is nice, but integrity is critical
- Encryption provides **confidentiality**
 - prevents unauthorized disclosure
- Encryption alone does **not** assure integrity
 - recall one-time pad and attack on ECB

MAC

- Message Authentication Code (MAC)
 - Used for data **integrity**
 - Integrity is **not** the same as confidentiality
- MAC is computed as **CBC residue**
 - Compute CBC encryption, but only save the final ciphertext block



MAC Computation

- MAC computation (assuming N blocks)

$$C_1 = E(IV \oplus P_1, K),$$

$$C_2 = E(C_1 \oplus P_2, K),$$

$$C_3 = E(C_2 \oplus P_3, K), \dots$$

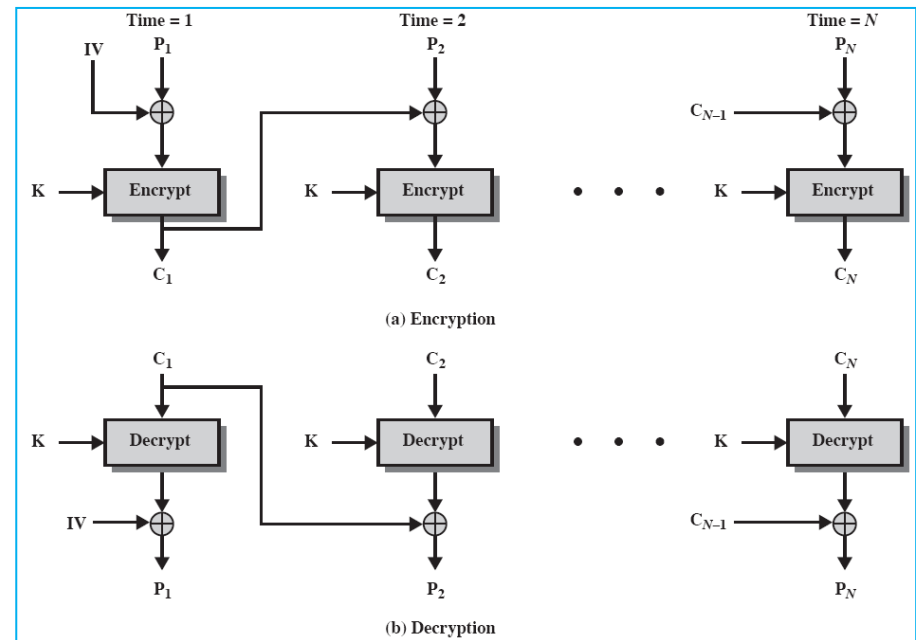
$$C_N = E(C_{N-1} \oplus P_N, K) = \text{MAC}$$

- MAC is sent along with plaintext

- Receiver

- Do the same computation and verifies that result agrees with MAC

- Receiver must also know the key K



Why does a MAC work?

- Suppose Alice has 4 plaintext blocks
- Alice computes

$$C_1 = E(IV \oplus P_1, K), C_2 = E(C_1 \oplus P_2, K),$$

$$C_3 = E(C_2 \oplus P_3, K), C_4 = E(C_3 \oplus P_4, K) = \text{MAC}$$

- Alice sends IV, P_1, P_2, P_3, P_4 and MAC to Bob

- Suppose Trudy changes P_2 to X
- Bob computes

$$C_1 = E(IV \oplus P_1, K), \textcolor{red}{C}_2 = E(C_1 \oplus \textcolor{blue}{X}, K),$$

$$\textcolor{red}{C}_3 = E(\textcolor{red}{C}_2 \oplus P_3, K), \textcolor{red}{C}_4 = E(\textcolor{red}{C}_3 \oplus P_4, K) = \textcolor{red}{MAC}' \neq \text{MAC}$$

- Error **propagates** into **MAC'** (unlike CBC decryption)
- Trudy can't compute **MAC'** without key K

Confidentiality and Integrity

- Encrypt with one key, compute MAC with another
- Why not use the same key?
 - Send last encrypted block (MAC) twice?
 - Can't add any security!
- Use different keys to encrypt and compute MAC
 - Even if keys are related
 - But still twice as much work as encryption alone
- Confidentiality and integrity with one “encryption” is a research topic

Uses for Symmetric Crypto

- Confidentiality
 - Transmitting data over insecure channel
 - Secure storage on insecure media
- Integrity (MAC)
- Authentication protocols (later...)

HW

- Java 기반의 AES 복호화 프로그램 작성
 - 아래의 key, IV, ciphertext를 이용해 plaintext를 구하시오
 - 각 값들은 Base64 인코딩되어 제공됨
 - Base64 인코딩을 공부해보자
 - Key : 8iE3bf1se6N76HGPP8S0Xw==
 - IV : cHml3oX848/0uBwDJtChOA==
 - C :
QDr9NZNG9Bgc3TTnfRuqjjzf/kVSYwbP7F9mR4GQZ/Inelh7HTc/xnwzEeVBc
H3pPIlbLFySKZruedJc9X87CGNDJ1f2Dat8BR3Ypbei5Q42xc306/AkSuGsjfqB
X9/ELxmdKn7MyvY/Jbc0v0AJHV6odgNzygKRRrFJcUIF/50=
 - 암호화 모드는 AES/CBC/PKCS5PADDING 임
 - PKCS5PADDING에 대해 조사해보자
 - Java JCA/JCE를 활용해 작성
 - <https://docs.oracle.com/en/java/javase/13/security/java-cryptography-architecture-jca-reference-guide.html#GUID-2BCFDD85-D533-4E6C-8CE9-29990DEB0190> 참조
 - 제출
 - 복호화된 plaintext & 구현한 code